

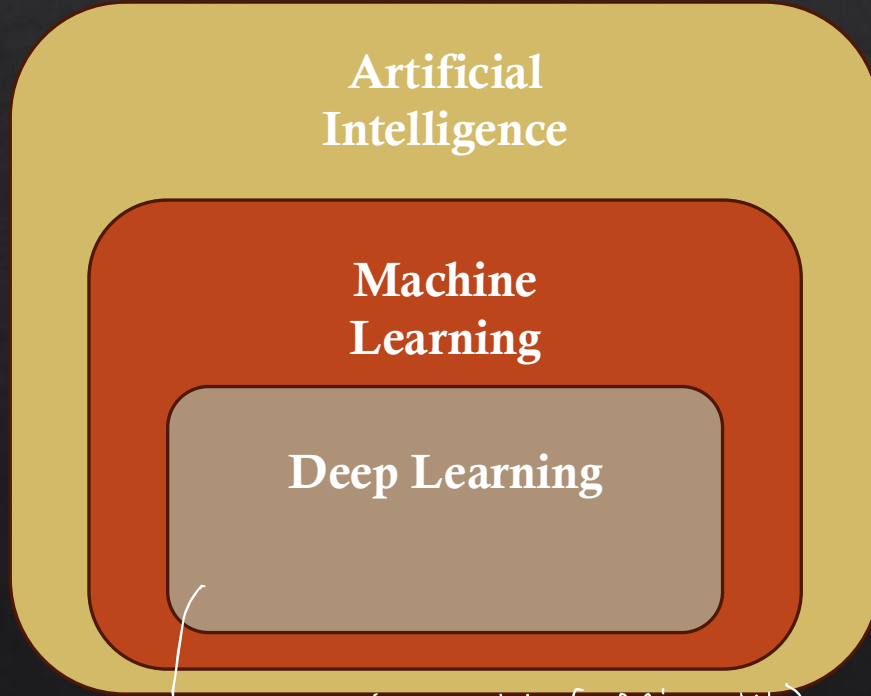


11. AI in Defensive Cybersecurity

Larry Huynh, S1 2026
larry.huynh@uwa.edu.au

Artificial Intelligence? Machine Learning?

uses neural network



↳ about the model (running ML itself)

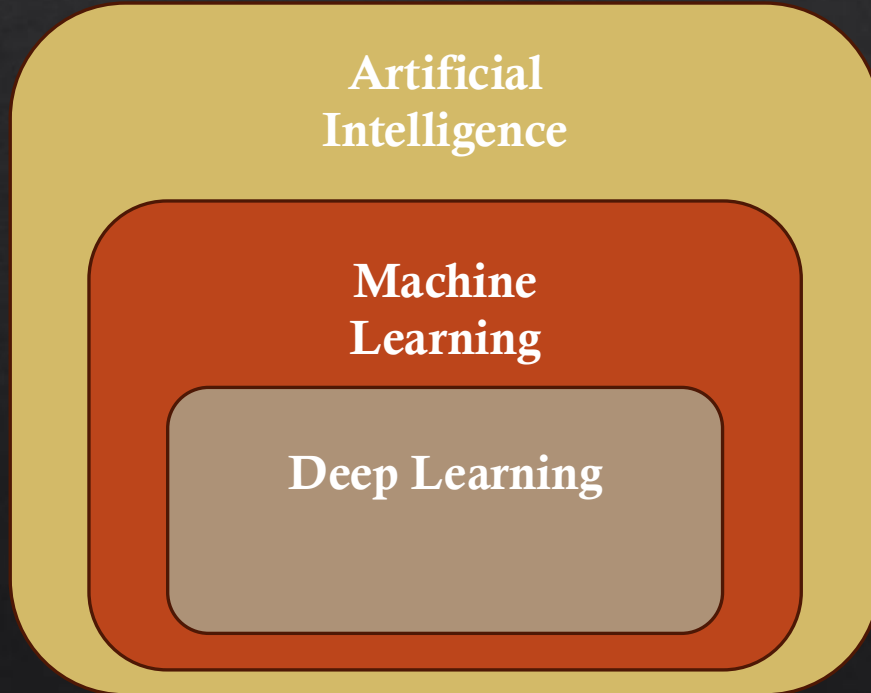
Artificial Intelligence

- The **theory** and **development** of **computer systems** able to perform tasks normally requiring human intelligence;
- Enabling a machine or system to sense, reason, act, or adapt like a human being.

↳ In the past, would've done it rule-based (heuristic)

↳ but now best way use **ML**
dump lots of data and ask AI to learn in autonomous way

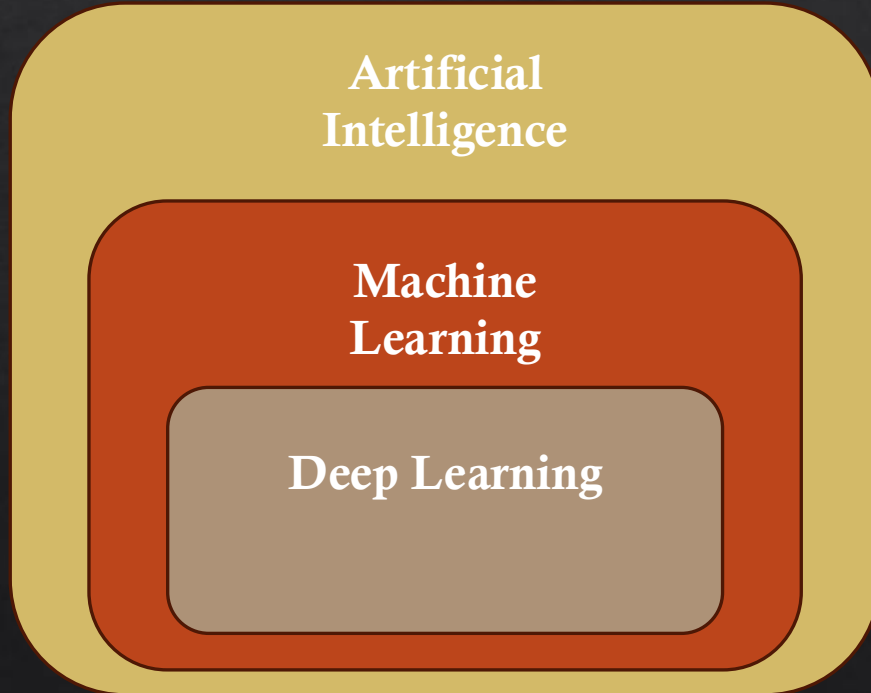
Artificial Intelligence? Machine Learning?



Machine Learning

- A means of producing/developing AI.
- Using machines to **extract knowledge from data and learn from it autonomously.**
- The computers “learn” without being explicitly programmed to do so.

Artificial Intelligence? Machine Learning?



Deep Learning

- A **subset of ML** that uses **large-scale artificial neural network** models.
- These try to **mimic the learning process of the human brain**.
- Often requires large amounts of **data** and **specialised GPUs** for training.

Machine Learning Pipeline: Training

1. Decide what you want machine to do

2. Get data in relation to that task
↳ choose & preprocess data
(remove noise, detect data)

Data Collection

Training Dataset



[Pizza] [Apple] [Grapes]

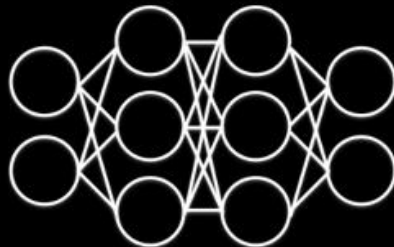


...

[Hotdog]



Machine Learning Model



"algorithm"
Training the Model

Evaluation



[Orange]

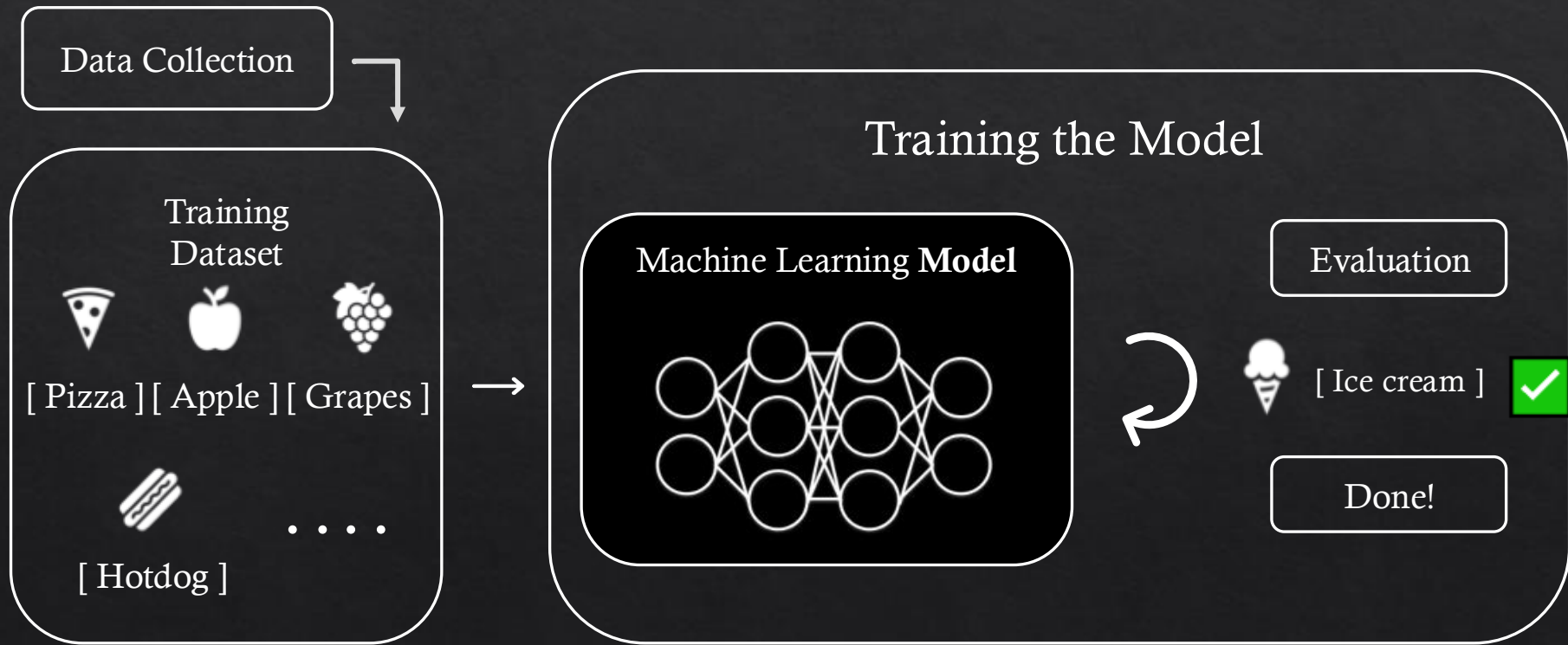


Re-training



iterative = looks @ some chunk of data,
learn patterns from data → learns from it, then go to
evaluation stage → retrain it not correct

Machine Learning Pipeline: Training

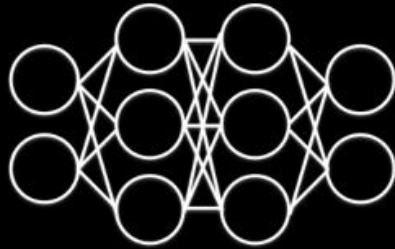


Machine Learning Pipeline: Predictions

New Instance



Trained Model



Classification



Model's Prediction

[Apple]

If model trained well,
should be quite good
@ prediction
for our context

model attempts to
classify image given

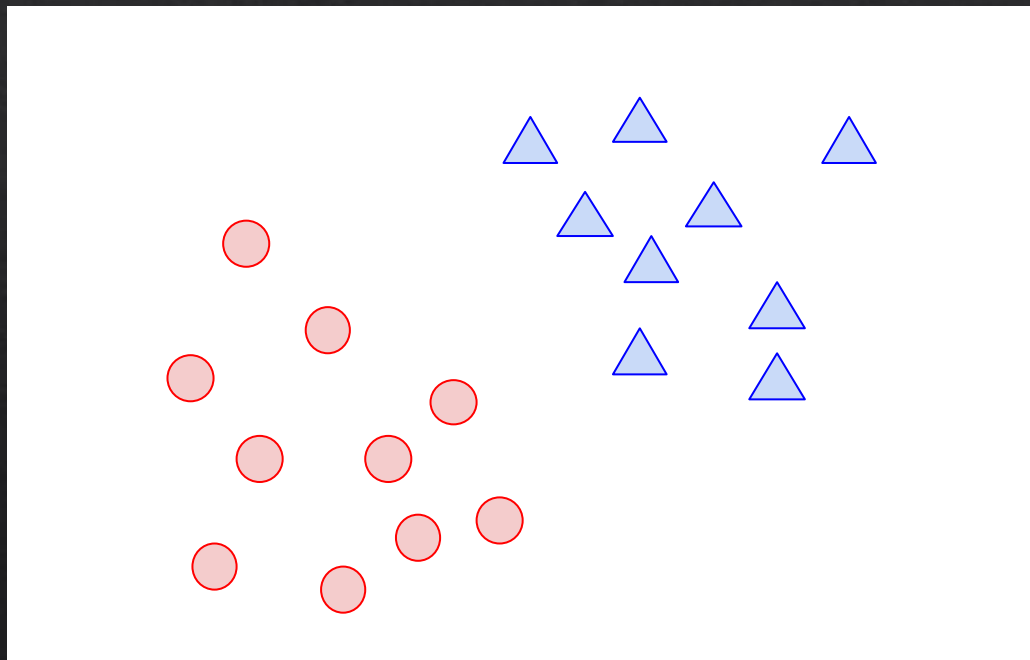
Machine Learning Models

A **model** learns useful patterns in data to achieve a task.

What and how it learns depends on the:

- Type of **problem**
- Available **information**
- Type of **model** used

Feature (Data) Space



Machine Learning Models

Supervised Learning

- Data is explicitly labelled
- E.g.

▲ = legitimate emails

● = spam

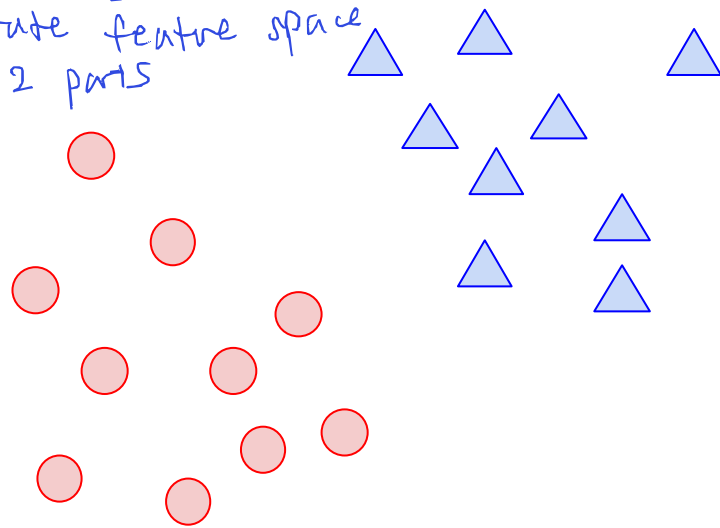
↳ do we know what each data point is about?

↳ know @ training time what data is what

↳ model can more accurately state @ inference time what data is what

• math representation of where data lies
• lies on 2 axis?
Feature (Data) Space plot data

rely on position of data
- separate feature space into 2 parts



x no of special written char perhaps? →

Machine Learning Models

Supervised Learning

- Data is explicitly labelled
- E.g.

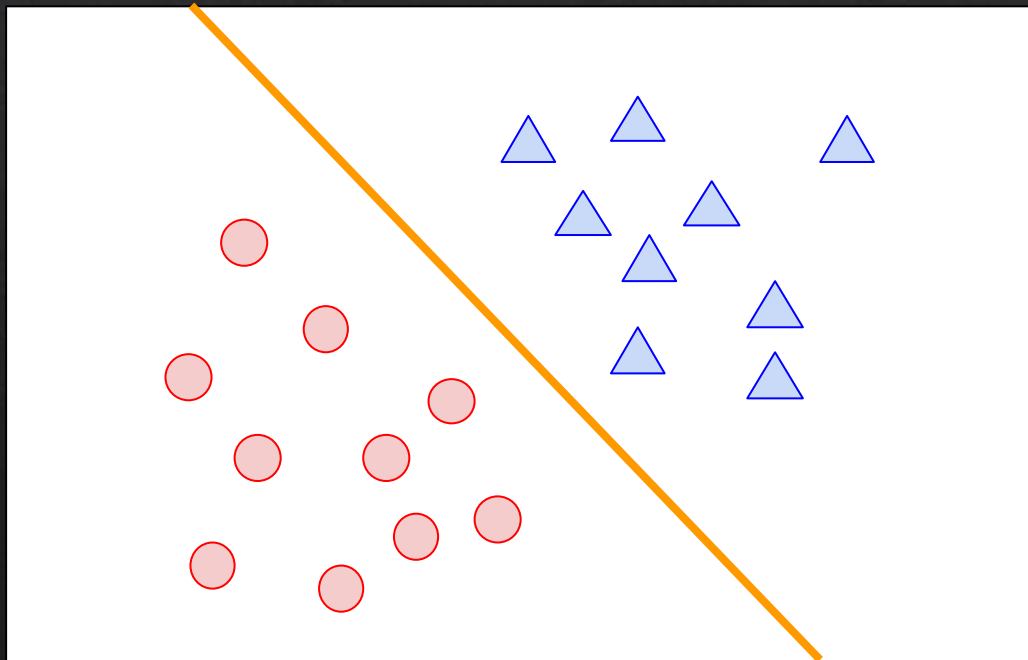
▲ = legitimate emails

● = spam

Decision Boundary (linear)

- separates features space into 2 parts based on what model has learn

Feature (Data) Space



Machine Learning Models

Supervised Learning

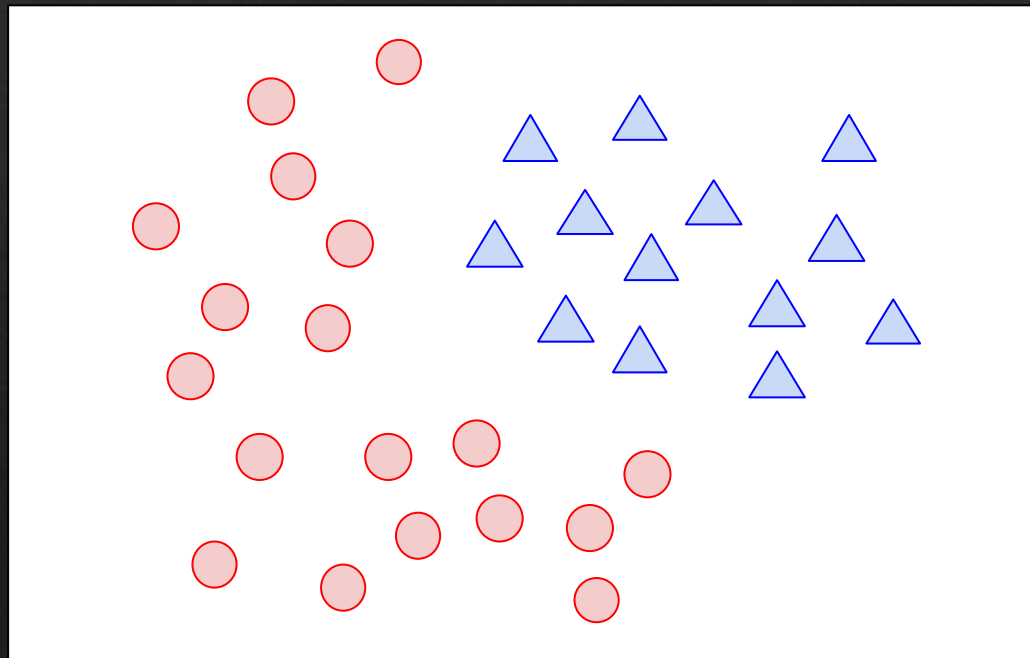
- Data is explicitly labelled
- E.g.

▲ = legitimate emails

● = spam

Decision Boundary?

Feature (Data) Space



Machine Learning Models

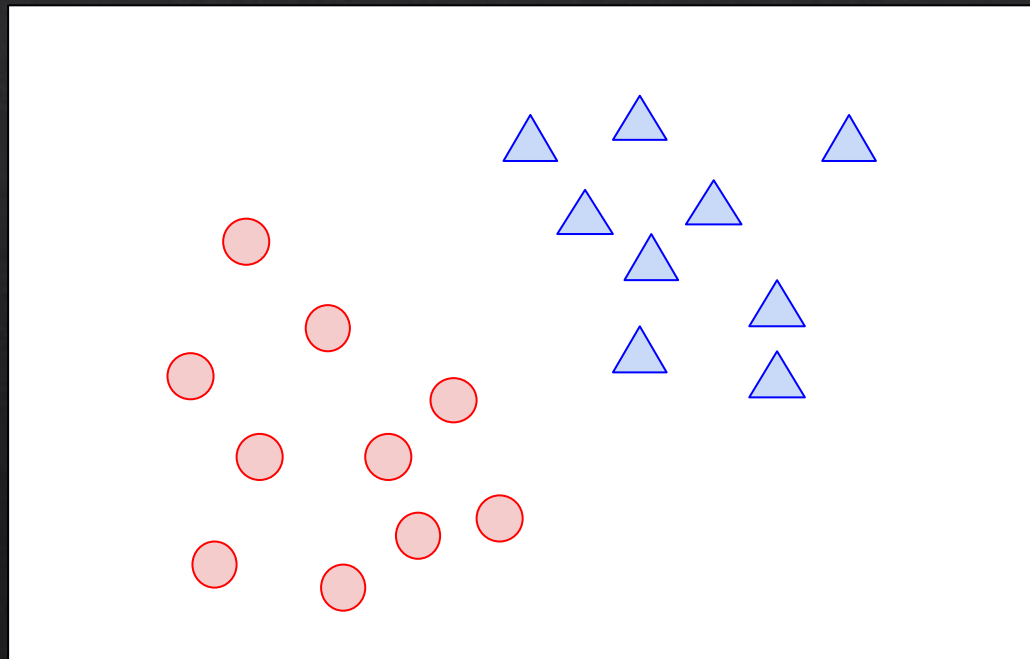
Supervised Learning

- Data is explicitly labelled
- E.g.

▲ = legitimate emails

● = spam

Feature (Data) Space



Machine Learning Models

- closeness of each cluster of data

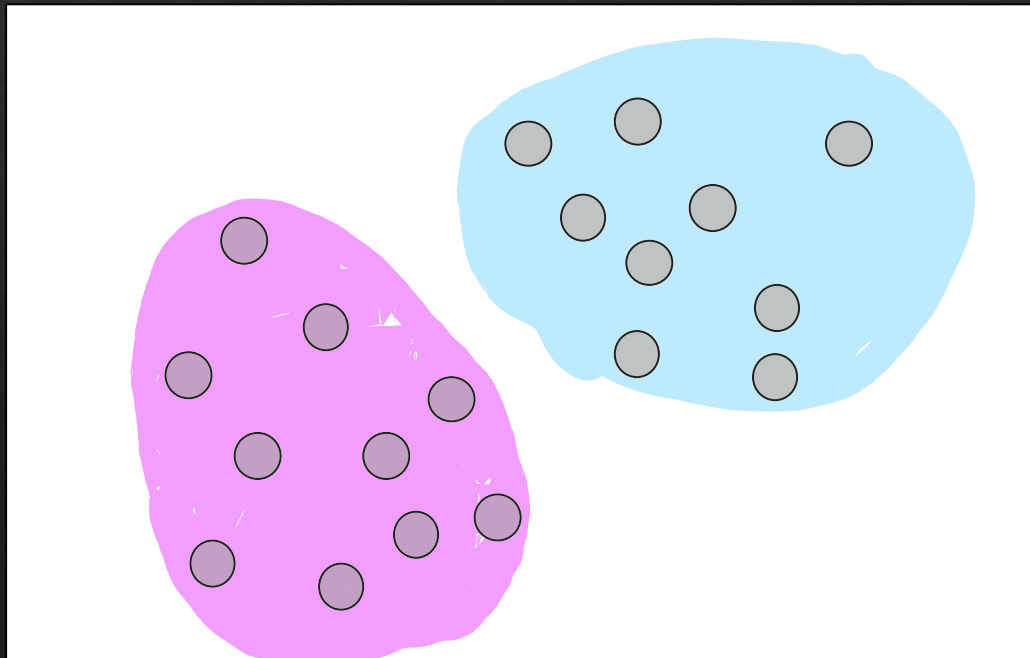
Unsupervised Learning

- Data is not labelled
- There are still emails and spam, but...

↳ at training time what is what

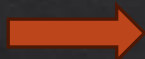
Clustering Algorithm

Feature (Data) Space



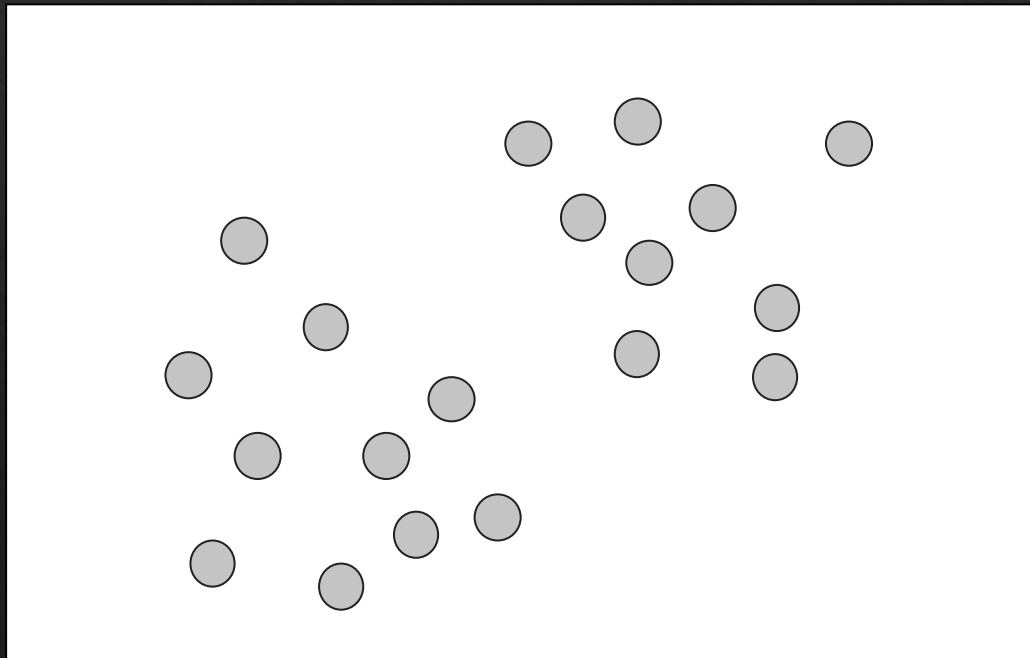
Machine Learning Models

Where does this feature
space representation of the
data come from?



Feature (Data) Space

Illustrate with an IDS
example.



Anomaly-based IDS

- ◇ Statistical anomaly-based IDS sample network activity, compare to “known normal” traffic
- ◇ IDS sounds alarm when activity is outside baseline parameters
- ◇ Advantage:
 - ◇ IDS can detect new types of attacks
- ◇ Disadvantages:
 - ◇ Requires more overhead, compute power than signature-based IDSs
 - ◇ May generate many false positives

also know the signatures of known malware

↳ legitimate data being flagged as anomaly

Anomaly-based IDS

↳ unsupervised, so no context / label

◇ Demo! ([https://github.com/uwacyber/cits2006/blob/2024/cits2006-labs/files/Anomaly based IDS.ipynb](https://github.com/uwacyber/cits2006/blob/2024/cits2006-labs/files/Anomaly%20based%20IDS.ipynb))

- Dataset = simulation of Mirai malware on simulated constructed network
= run for a period of time normally traffic, then run malware
- Wireshark extract meta data into (frame time, source/dest IP, TTL,)
↳ no payload info (depends on context)
↳ packet header info
- look @ communication
 - time-based packets
 - end-to-end (Flows) → packets go from one end to another, how to plot the info
 - stats-based features = computes stats at certain intervals

if good, error is small

Anomaly-based IDS

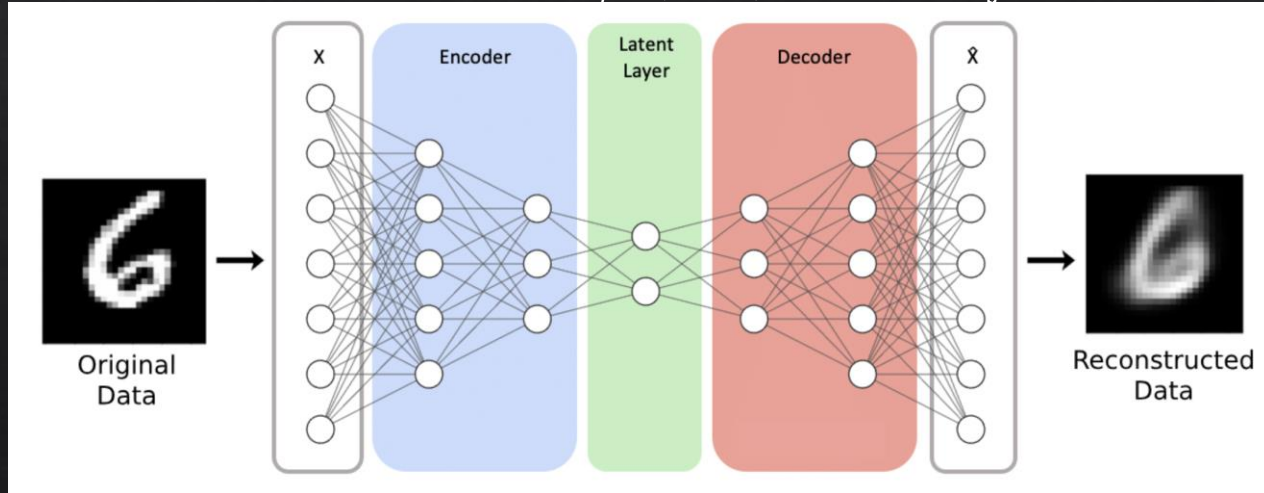
give model of data, feed through model, spits out model's reconstructed output

- ◇ Autoencoder Model

- ◇ This neural network architecture tries to learn the important features of your data by using a reconstruction-based learning process.

- ◇ The idea is that the model compresses (encode) the input down to a much smaller representation at the latent layer, then decompresses (decode) it to its original dimensions at the output layer.

specify how small you want to decompress it into

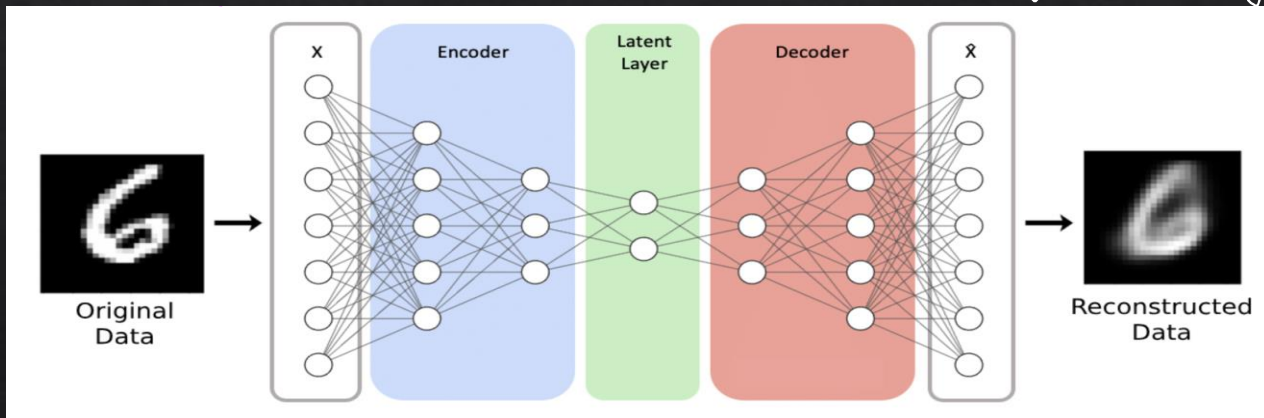


successful if can output good
model is able to take good into from data
(strong representation)

- Define input size and hidden size (encode)
 - Take smth from input size and output as input size (decode)
Anomaly-based IDS
 label 0 = normal data
 label 1 = anomalous data model
 - scale all data to "uniform" size so one data doesn't dominate & confuse the

◇ Autoencoder Model $loss = reconstruction\ error$ (want to be low)

- ◇ The goal is to essentially reconstruct the original image; for the model to successfully do this, the latent layer must have learnt the important characteristics of the input data.
- ◇ The model's ability to perform this reconstruction is characterised by the **reconstruction error**, which is just a number that measures how close the reconstruction is to its input—a low reconstruction error means a good reconstruction, while a high error means a poor reconstruction.
 ↳ may be anomaly



Anomaly-based IDS

- ◇ Different feature representations for the same data can produce different results:
 - ◇ using just the raw packets (Packets)
 - ◇ using a stats-based aggregate of the packets (Stats)
 - ◇ using end-to-end communication flows (Flows)
- ◇ Performance of encoding methods using autoencoder on the Mirai BotNet IDS dataset (from demo):

may not represent fully the data (one class might do better than others ?)

	Raw	Flows	Stats
Accuracy	88.77% ~	43.59% 'n'	98.80% 'v'
Precision	98.54%	39.47%	100.00%
Recall	87.94%	88.34%	96.79%
F1	92.94% ~	54.56% 'n'	98.37% 'v'

help quantify
accuracy
effect in
case value
is misleading

Easier to separate data if it's well represented?
→ easily separable

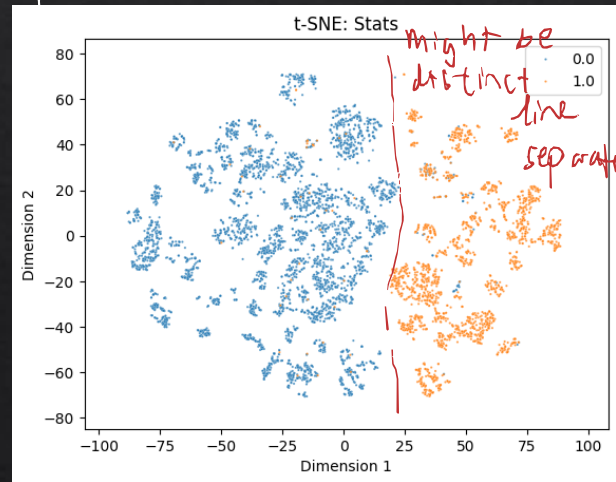
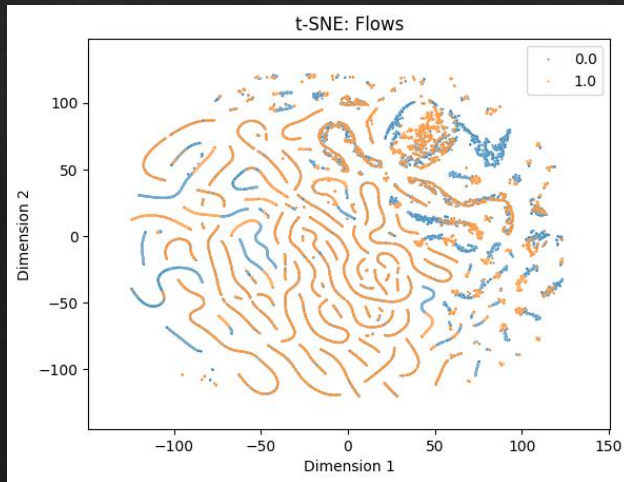
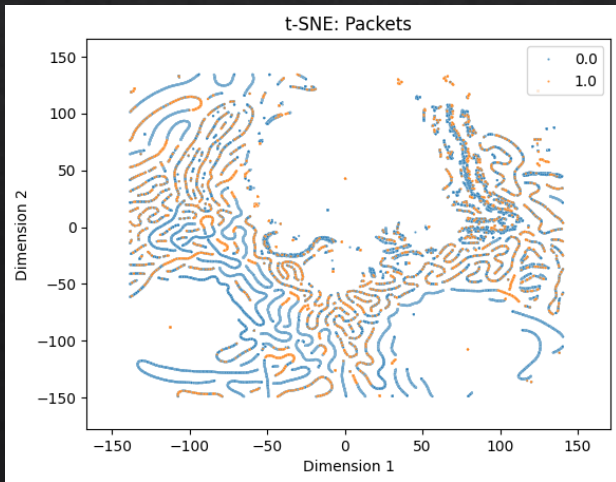
Anomaly-based IDS

blue = normal data ; orange = anomaly data

(0) [benign] (1) [Malicious]

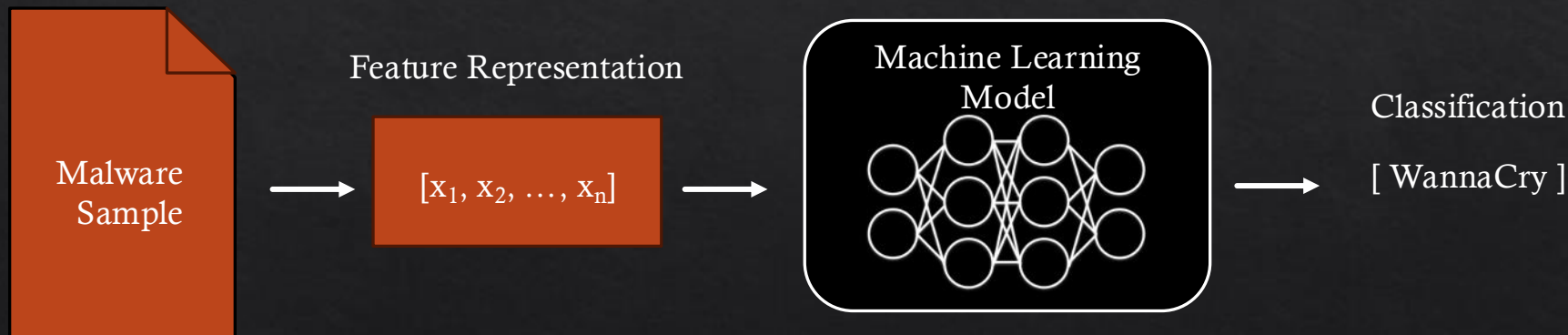
- ◇ The t-SNE provides a reduced dimensional view of the given data.
- ◇ t-SNE plots for the Packet, Flow and Stats datasets below.
- ◇ As we can see, different encodings may induce varying degrees of separability, which may be reflected in the machine learning performance metrics.

→ better spread & clustering



Malware Detection

- ◇ Using machine learning to:
 - ◇ Detect malware files (or the presence of malware within files) and/or
 - ◇ Classify and assign a malware sample to a specific malware family



Malware Detection

◇ How to represent malware signatures as features for machine learning?

◇ String Analysis

- ◇ Extracting printable strings from malware executables

◇ Byte and Opcode N-grams

- malware - arbitrary sys call
- weird movement around memory
- ◇ Looking at n-contiguous byte sequences, or assembly language instructions

◇ API Function Calls/Function Call Graphs

- for some exfiltration task
- ◇ Particular API functions can provide key information for representing malware behaviours
- ◇ These can also be modelled as a directed graph; edges symbolise function calls, nodes represent functions

◇ Malware Representation as a Grayscale Image

- binary representation of a file = 0 or 1
- ◇ Interpret every byte as a pixel in an image (values ranging from 0 - 255)
- ◇ Resulting array organized as a 2D image
- byte form values same as img format

Malware Detection

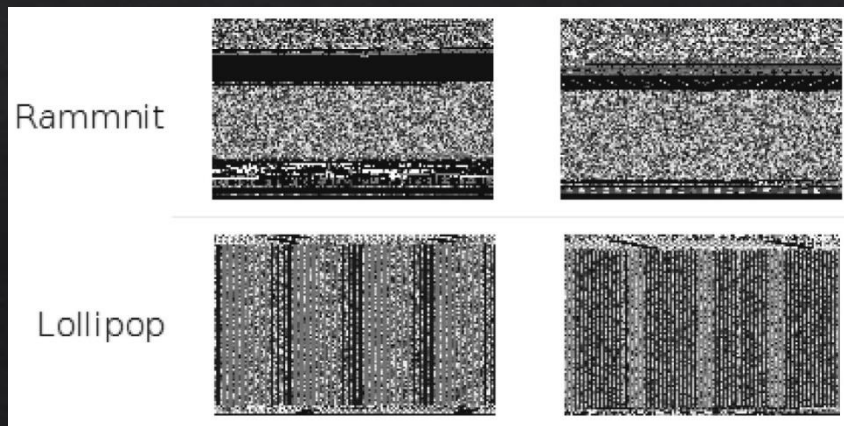
- ◆ Malware Representation as a Grayscale Image

- ◆ Interpret every byte as a pixel in an image (values ranging from 0 - 255)
- ◆ Resulting array organized as a 2D image

- ◆ Malware samples from the same family tend to exhibit similar textural patterns

- ◆ Neither disassembly nor code execution is required for classification!

→ no need to look into the files itself



validation = can think as mother test
train = malware families

Malware Detection

- ◇ Demo! (<https://github.com/uwacyber/cits2006/blob/2024/cits2006-labs/files/Malware-detection.ipynb>)

CoAtNet

- "Transformer" = self-attention to capture global interactions
= take correlations that occur, retain it

As you go through
convolution may lose
the pixel correlation/
global interaction

- Confusion matrix
= tells you visually how well you did on classes
= highly performing model should have strong diagonals
- ↗ If given input A,
expect output A
as well

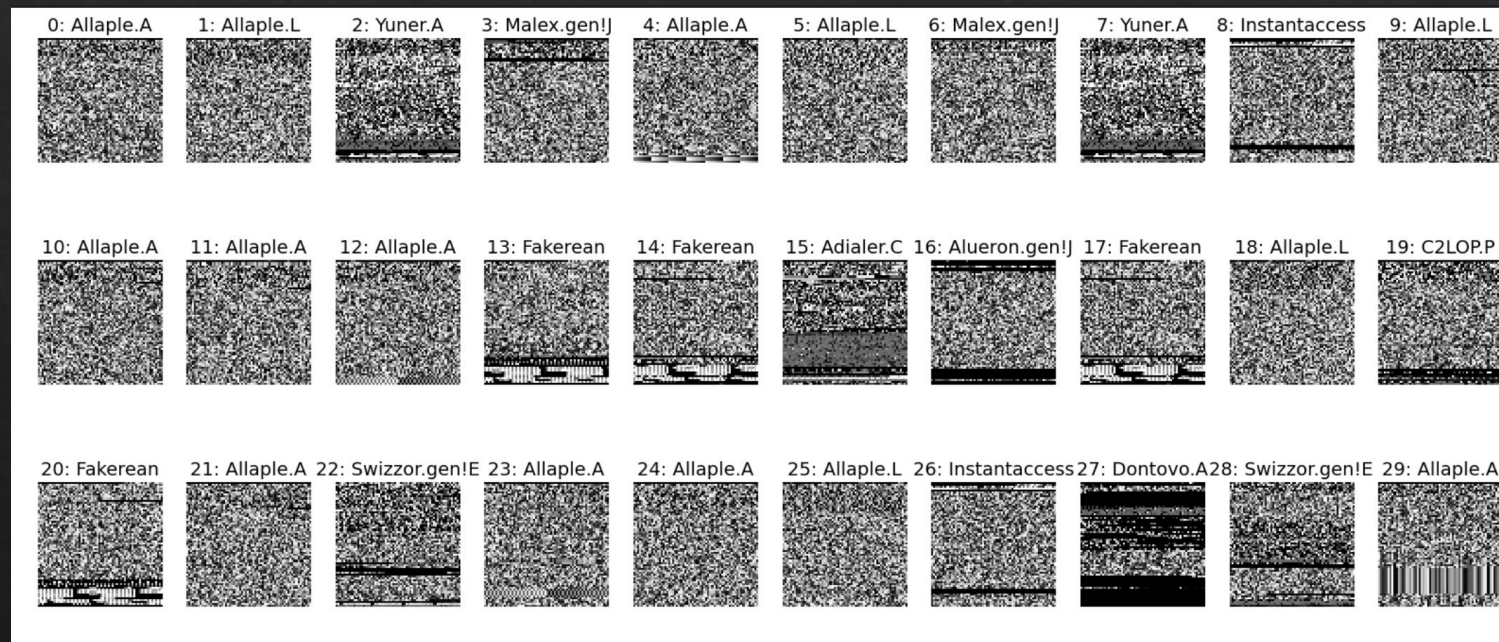
Malware Detection

histogram shows skew for

Allaple?

data (~ 15% ?
50% ?)

- ◆ The MalImg-dataset-9010 dataset of 25 malware samples, each converted from their binary representations into 2D grayscale images:



weights & biases, each node in each wire? is parameterized

Malware Detection

fit() = function used to train model

"training" = updating parameters on graph

from grid structure to
"Output layer" class 1, class 2... -

- ◇ We can use machine learning over this dataset to detect different types of malware. take output array & readjust its size to be fit for
- ◇ As we saw in the demo, different machine-learning models can provide various performance levels, even on the same encoding. max pooling layer

◇ Feed-Forward Neural Network (FFNN): 87.31% accuracy

- ◇ Not able to capture short and long-range structural correlations within the data. Images will tend to exhibit this sort of correlation (i.e. pixels near each other in an image are probably related).

◇ Convolutional Neural Network (CNN): 97.15% accuracy

- ◇ Can capture the above correlations; it seems malware as images also exhibit structure correlations.
- ◇ However, long-range correlations are still an issue.

◇ Convolutional Attention Neural Network (CoAtNet): 98.18% accuracy

- ◇ Combines CNNs with state-of-the-art Transformer networks to capture long-range correlations, leading to the best performance.

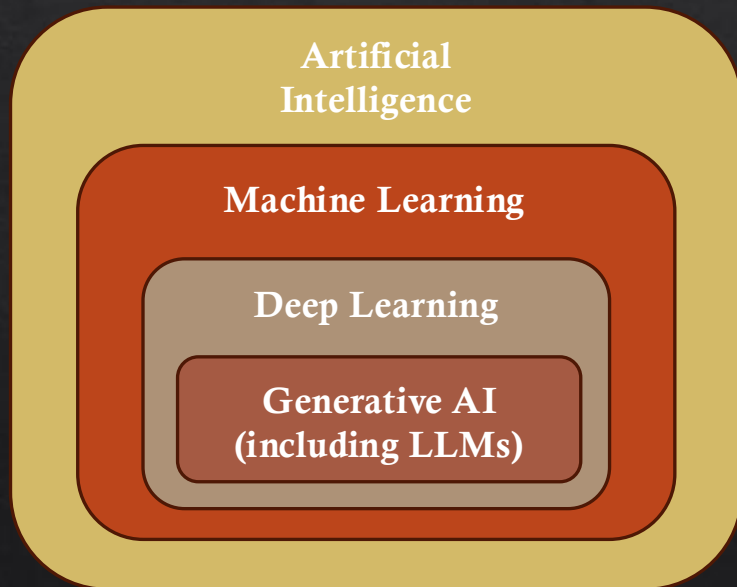
"Convolution" = scan across image, take a segment of it (filter), then compute a summary and put into array

depends on type of img
* also filter
extracts horizontal & vertical line into

next convolution layer

↳ "premier" model for image modelling?

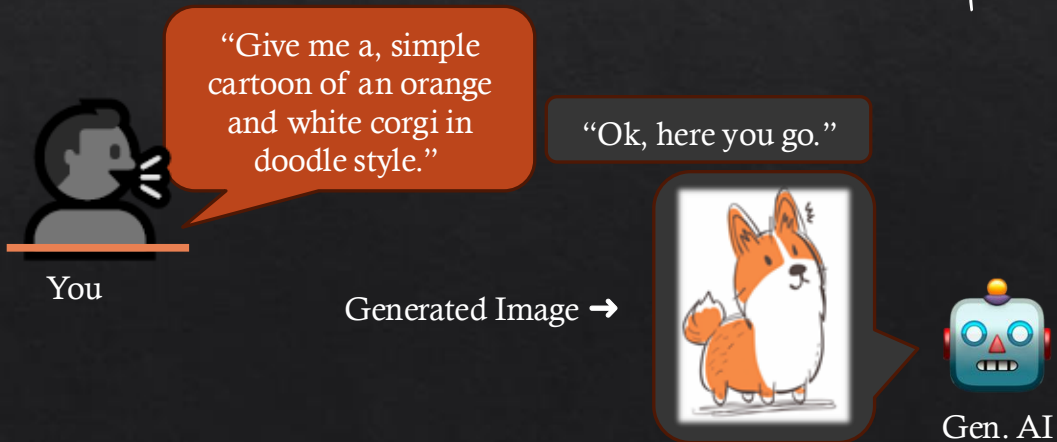
Large Language Models (LLMs)



◇ LLMs are a part of the new wave of **generative AI models**:

◇ Instead of the model outputting a single (classification) result, gen. AI models can generate new responses (images, text, etc.)

unstructured response



Large Language Models (LLMs)

◇ LLMs are:

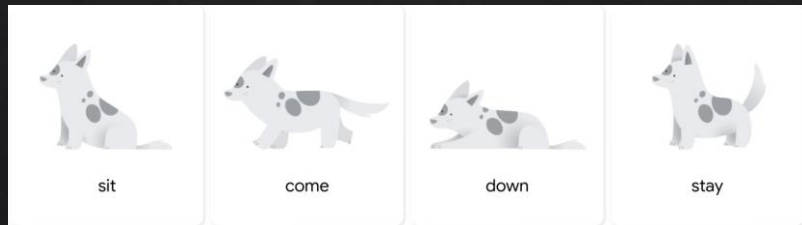
- ◇ **Large**: huge training datasets, and huge model parameters used to develop them.
- ◇ **General purpose**: the models are sufficient at solving a wide range of common problems.
- ◇ **Often Prompt-based**: you interact with the model by giving it a user input query.

◇ **Pre-trained and fine-tuned.**

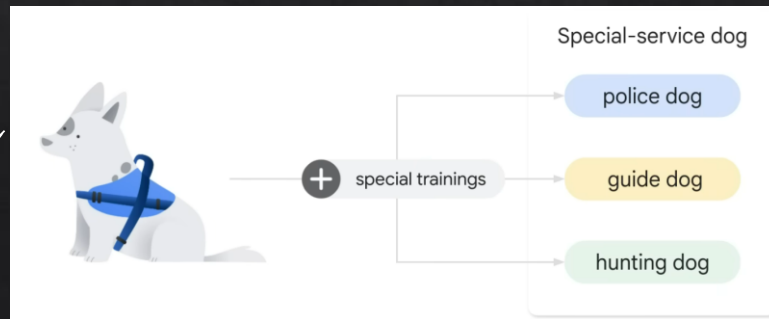
= train LLM model on generic textual data
= it have smth that's general purpose,
"Pre-trained" want to hone in on smth specific

model size = no. of parameters

e.g. grammar, syntax
(allow it to speak correctly)
"Fine-tuned"



General Purpose Dog



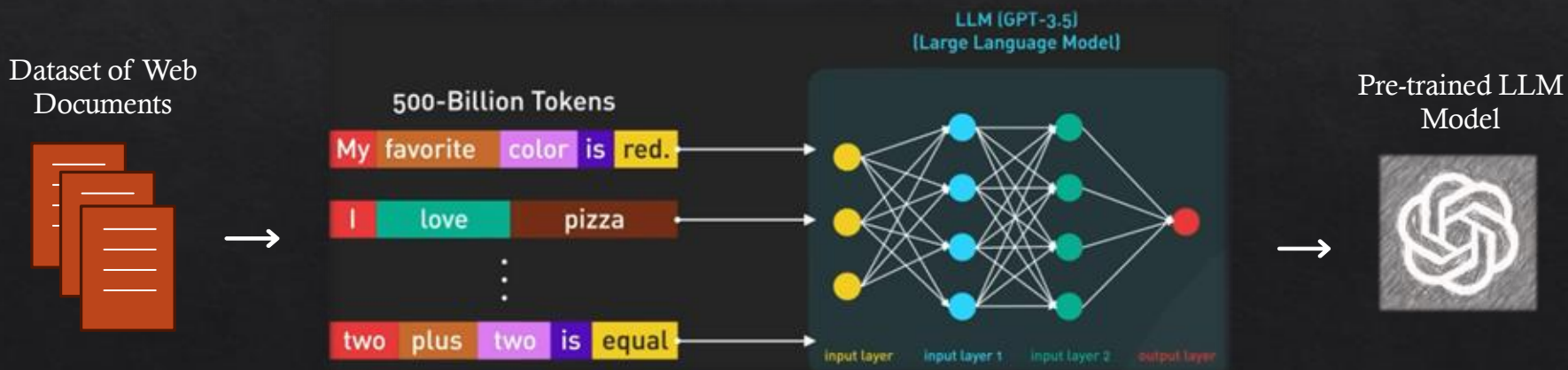
Special Purpose Dog

Large Language Models (LLMs)

◆ 1. Pre-training:

- ◆ The model learns general language representations.
- ◆ By training on diverse text data, LLMs can capture rich linguistic information and learn meaningful representations of words, phrases, and sentences.

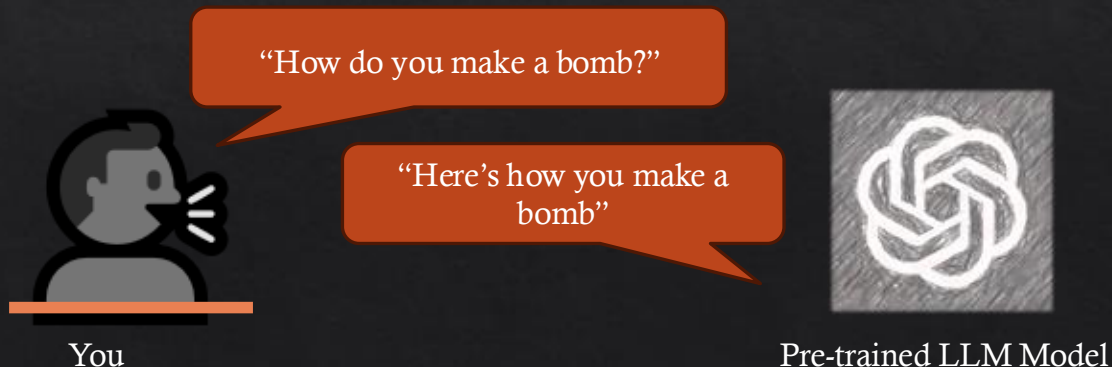
Pre-Training



Large Language Models (LLMs)

◇ 1.5. Safety Training:

- ◇ After pre-training, we now have an LLM that understands general prose, grammar, etc.
- ◇ The LLM learns to “talk” like a human.
- ◇ However, since it's been trained on web data, it may generate impolite, untruthful, or even harmful information.



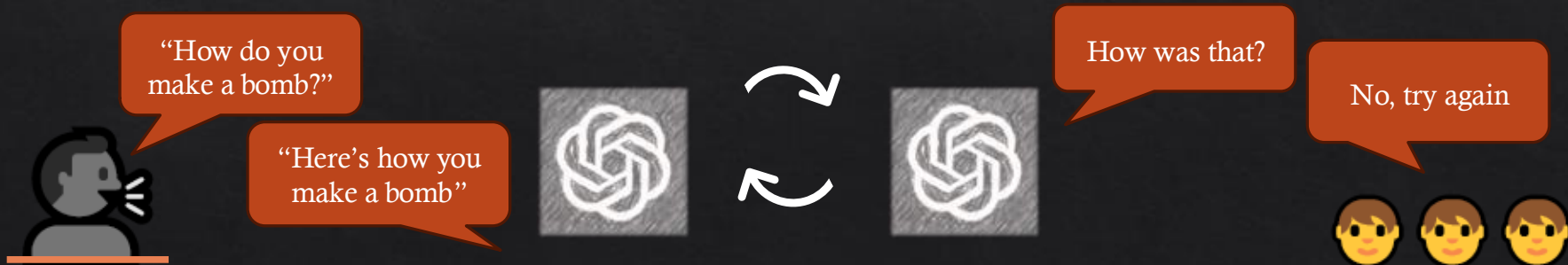
Large Language Models (LLMs)

◇ 1.5. Safety Training:

- ◇ Solution: Reinforcement Learning with Human Feedback (RLHF)
- ◇ Further train the model using human preferences and feedback as a reward signal;
- ◇ It involves humans providing feedback on the quality of an AI's outputs, which is then used to fine-tune the model to better align with human preferences.

steers more towards
safe & preferred signal

RLHF Training Loop



Large Language Models (LLMs)

◇ 1.5. Safety Training:

- ◇ Solution: Reinforcement Learning with Human Feedback (RLHF)
- ◇ Further train the model using human preferences and feedback as a reward signal;
- ◇ It involves humans providing feedback on the quality of an AI's outputs, which is then used to fine-tune the model to better align with human preferences.



Large Language Models (LLMs)

- ◆ LLM Refusals and Dual-Use Cybersecurity *helping you (model's goal)* vs. *being safe (following safety rules)*
 - ◆ refusal training can over-trigger on legitimate work or under-trigger on disguised intent. Framing is often the model's only signal.

Prompt A: Harmful Intent

“Give me a step-by-step attack plan for compromising a login page.”

Expected Behavior:

- ◆ Refuse or redirect to lawful/defensive guidance

Prompt B: Ambiguous Cyber Request

“I have permission to test a login page. Give me exact payloads and step-by-step attempts to bypass the login.”

Expected Behavior:

- ◆ May refuse, hedge, or give cautious, scope-aware guidance

Prompt C: Defensive Framing

“I'm pen-testing my own login page in a local lab setup. What are the main vulnerability classes I should check, and how do I test for each?”

Expected Behavior:

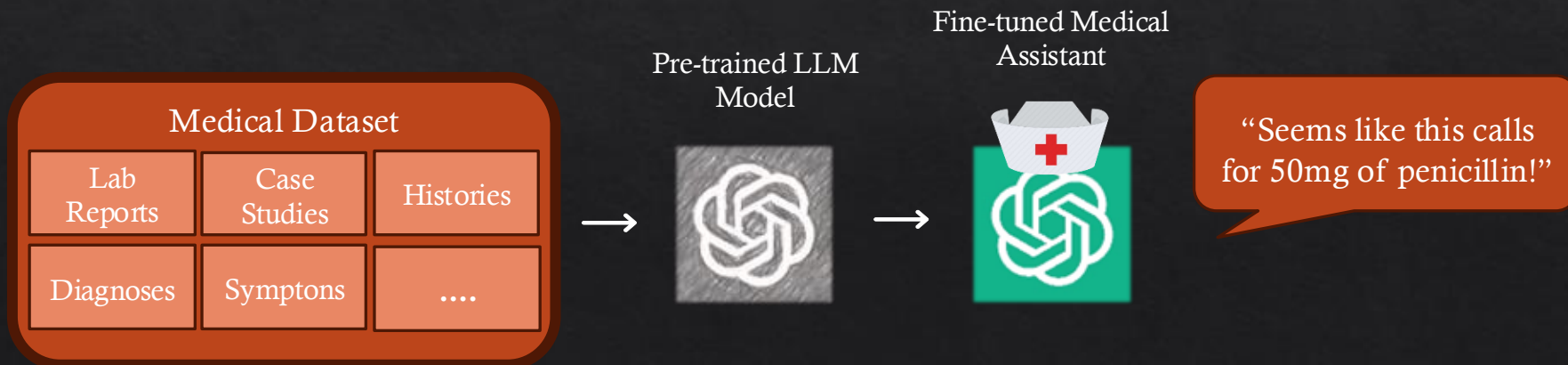
- ◆ More likely to provide safe defensive guidance.

Large Language Models (LLMs)

◇ 2. Fine-tuning:

- ◇ Adapting the model to a specific domain or task.
- ◇ Using the learned parameters of the pre-trained model as a starting point, retrain the model on a (usually much smaller) domain-specific dataset. For example, LLMs for diagnosis:

Fine-Tuning



Code Vulnerability Detection (CVD)

- ◆ **The problem:** find security bugs in source code *before* they ship.
 - ◆ e.g. buffer overflows, use-after-free, SQL injection, command injection.
- ◆ Traditional approaches:
 - ◆ Manual code review: thorough but slow and error-prone.
 - ◆ Static analysis (e.g. Coverity, CodeQL): fast and rule-based, but misses context-dependent bugs.
- ◆ Why try LLMs?
 - ◆ Trained on huge code corpora; can reason about intent, not just patterns.
 - ◆ No rule-writing required; just describe the task.
- ◆ **But how good are LLMs at CVD?**

How Much Do We Trust LLMs for Coding?

From a survey of 1,470 developers (“Detecting Code Vulnerabilities using LLMs (2025)”):

97% kept using LLMs after their first try.

82% express at least some trust in LLM-generated code.

80% are satisfied with LLMs at finding/fixing vulnerabilities.

◆ Users may be over-reliant on LLMs and overestimate the security of their outputs.

Asking an LLM to find a Simple Bug

Prompt: "Is this code safe? If not, what's wrong with it?"

```
void greet(char *name) {  
    char buf[16];  
    strcpy(buf, name);  
    printf("Hello, %s\n", buf);  
}
```

Handwritten annotations:

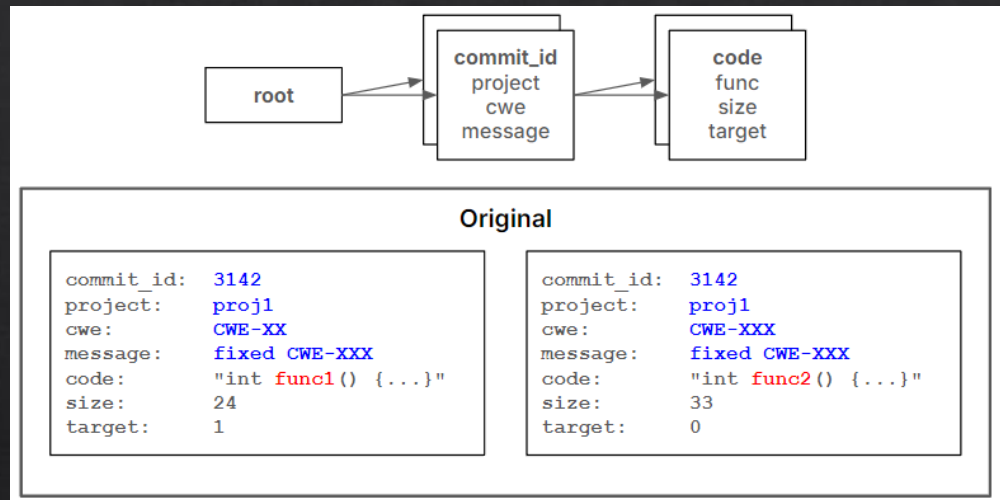
- memory* (pointing to the opening curly brace of the function body)
- buffer overflow* (above the `strcpy` line)
- invalidate with input, memory isn't allocated for buffer* (under the `strcpy` line)

More Complex Code Contexts

- ◇ Simple code contexts may not represent **real workflows and use cases**.
 - ◇ Large codebases with unstructured code
 - ◇ Vulnerabilities can span multiple functions/files
- ◇ Programmer may not be able to easily isolate the affected code, or easily know if code is vulnerable at all.
- ◇ More likely usage patterns: chuck large chunks of code into the LLM.
- ◇ Do LLMs respond well in these scenarios?

More Complex Code Contexts

- ◇ We need to look at some real code!
- ◇ E.g. [DiverseVul Dataset](#):
 - ◇ Large C/C++ vulnerability dataset.
 - ◇ Built from real GitHub commits that fixed known vulnerabilities, linked to CWEs.
 - ◇ Vulnerabilities can span single and multiple functions and files.
 - ◇ Gives realistic code contexts, not just isolated toy snippets.



DiverseVul Dataset Structure

More Complex Code Contexts

◇ Example DiverseVul datapoints:

- ◇ Example 1: ISDN `isdn_net_setcfg`: unchecked `strcpy(drvid, cfg->drvid)` into a 25-byte buffer (CWE-119).
 - ◇ [Code](#), [ChatGPT5.5 Conversation](#)
- ◇ Example 2: OSS sequencer `snd_seq_oss_open`: error path double-frees `dp` via `delete_port`'s callback and a trailing `kfree` (CWE-415).
 - ◇ [Code](#), [ChatGPT5.5 Conversation](#)
- ◇ Example 3: `timerfd_create`: `kfree(ctx)` on `anon_inode_getfd` failure after `ctx` is already wired into a struct file, causing a use-after-free against the file's release path (CWE-416).
 - ◇ [Code](#), [ChatGPT5.5 Conversation](#)
- ◇ Example 4: OpenSSL `print_qualifiers` / `print_notice`: read-only ASN.1 printers with bounded `%.*s`, NULL-checked allocations, and no caller-controlled writes — no vulnerability.
 - ◇ [Code](#), [ChatGPT5.5 Conversation](#)

NOTE: You don't need to read the code closely, the takeaway is in the vulnerabilities and the LLM's responses

LLMs for CVD?

◇ Are LLMs good at CVD? It depends:

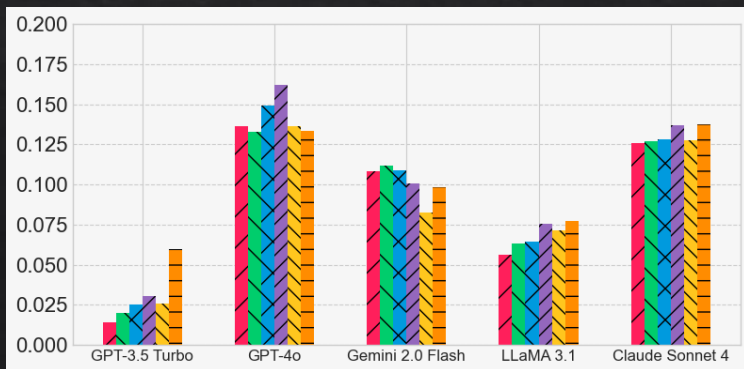
◇ If you want the LLM to strictly get the correct CWE, then not really (yet!) (F1: ~ 0.16)

◇ If you want to flag ANY vulnerability, then they perform quite well (human in the loop!) (F1: ~ 0.67)

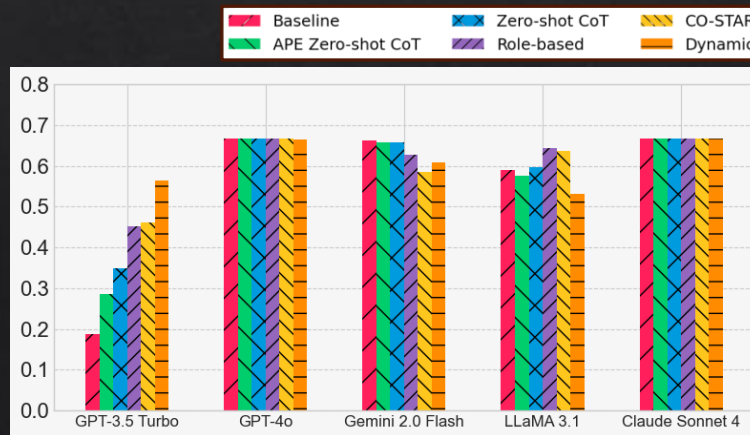
◇ *“Detecting Code Vulnerabilities using LLM (2025)”*

◇ 5 LLMs, 6 prompting techniques, on DiverseVul

F1
score



Correct CWE



Any CWE

LLMs for CVD?

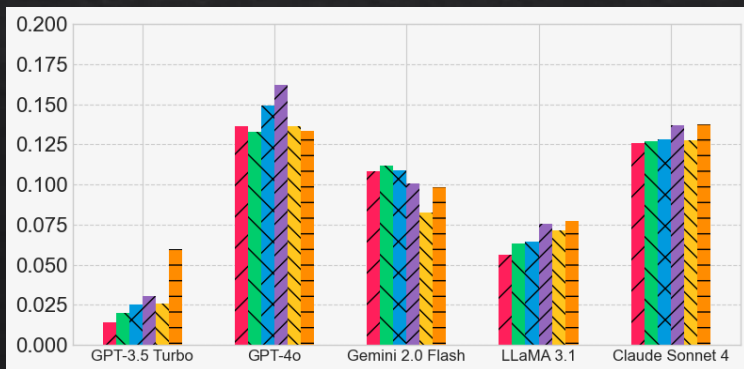
◆ Does prompt engineering matter?

- ◆ More so in weaker models, less so in more capable models.
- ◆ Model capability still matters: weaker models may give shallow, unstable, or overconfident security judgements. E.g. Qwen2.5-Coder-3B-Instruct conflicting responses ([R1](#) vs. [R2](#)).

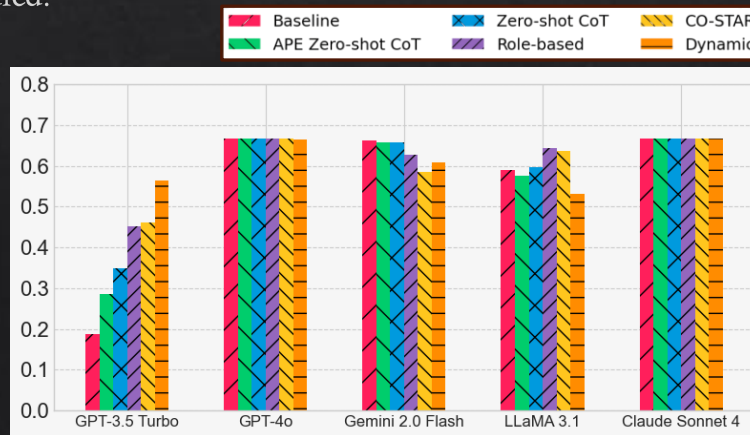
◆ What about newer models? (ChatGPT-5.5, Claude Opus 4.7)

- ◆ Likely much better, however yet to be empirically studied.

F1
score



Correct CWE



Any CWE

LLMs for CVD?

LLMs are useful, but not authoritative.

- ◆ They can flag suspicious code and explain possible security issues.
- ◆ They can still miss real vulnerabilities, misclassify CWEs, or invent problems.
- ◆ More code context does not automatically mean better answers.
- ◆ Prompting helps, especially for weaker models, but does not replace security understanding.
- ◆ Best use case: human-in-the-loop triage, not fully automated vulnerability judgement.

References

◇ Malware:

- ◇ https://vision.ece.ucsb.edu/sites/default/files/publications/nataraj_vizsec_2011_paper.pdf
- ◇ <https://www.sciencedirect.com/science/article/pii/S1084804519303868#sec4>
- ◇ <https://www.kaggle.com/code/gvyshnya/malimg-coatnet-model>

◇ LLMs

- ◇ <https://arxiv.org/pdf/2402.00891>
- ◇ <https://dl.acm.org/doi/abs/10.1145/3607199.3607242>
- ◇ <https://ieeexplore.ieee.org/abstract/document/11068842>